Perceptron Based Replacement Policy in GPU LLC

Rami Abo Mokh

***Abstract*—The latency gap between GPU last-level cache (LLC) and off-chip memory severely impacts throughput for memory-intensive workloads. Conventional replacement policies such as Least Recently Used (LRU) and PseudoLRU fail to predict reuse accurately under the irregular and massively parallel access patterns of modern GPUs. While prior CPU-oriented work, *Perceptron Learning for Reuse Prediction* (MICRO 2016), demonstrated that perceptron learning can provide superior reuse prediction by leveraging program counter (PC) features, this approach could not be applied directly in our experiments because the GPU simulator used (MGPUSim) does not expose PC information to the cache replacement policy.**

We propose a PC-free perceptron-based cache replacement policy that uses carefully selected address bits as a proxy for PC features, combined with a confidence-aware hybrid mechanism that falls back to PseudoLRU when prediction certainty is low. The predictor operates online, with lightweight training and sampling strategies to bound computational overhead. We integrate the design into the MGPUSim GPU simulator via a backward-compatible extension to the victim selection interface, and evaluate it across three workloads: sparse matrix-vector multiplication (SPMV), BiCG, and k-means clustering.

Results show 8–22% L2 miss reduction and 4–15% average latency improvement over LRU, with negligible storage overhead ($< 0.01\%$ of LLC) and computational cost ($< 1\%$). These findings demonstrate that machine-learning-driven reuse prediction can be adapted effectively even without PC access, offering a practical path toward more intelligent cache management in GPU simulation environments.

I. Introduction

Modern GPUs achieve high throughput by executing thousands of threads in parallel, which puts significant pressure on the memory hierarchy. The last-level cache (LLC) plays a critical role in hiding off-chip memory latency; however, its effectiveness is highly dependent on the cache replacement policy. Conventional policies such as Least Recently Used (LRU) or PseudoLRU make eviction decisions based solely on recency, which often fails under the irregular and interleaved access patterns common in GPU workloads. Ineffective replacement decisions lead to avoidable cache misses, increased average memory access latency, and reduced overall performance.

Recent research has explored more intelligent eviction policies that predict whether a cache block will be reused before it is evicted. One of the most notable approaches is *Perceptron Learning for Reuse Prediction*, proposed by Terán et al. in MICRO 2016 [1]. In that work, the authors introduced a hardware-friendly perceptron predictor that uses multiple feature tables indexed by program counter (PC) and address bits to produce a reuse prediction score. The perceptron learns online: when a prediction is incorrect or lacks confidence, the associated feature weights are updated using a simple bounded integer arithmetic. This method achieved a significant improvement over dead-block prediction and LRU, with average speedups of 6.1% and a false positive rate as low as 3.2%. The key insight of MICRO 2016 is that the PC encodes control-flow context that strongly correlates with memory access reuse, allowing the perceptron to adapt dynamically to changing program phases.

While the MICRO 2016 perceptron is highly effective in CPU systems, directly applying it to our experimental setting was not possible. The GPU simulator used in this study, MGPUSim [2], does not expose PC information to the cache replacement policy. This limitation required us to develop an alternative feature representation that does not rely on PC values, while retaining the perceptron's ability to learn reuse behavior online.

In this work, we propose a **PC-free perceptron-based cache replacement policy** that uses selected bits of the memory address as a proxy for the PC-based features of the original algorithm. Our design also incorporates a **confidence-aware hybrid** mechanism: when the perceptron's prediction confidence falls below a threshold, the policy falls back to a baseline PseudoLRU decision. To further bound computational overhead, we employ training sampling (20% of accesses) and lightweight state caching between prediction and training phases.

We integrate our design into MGPUSim via a backward-compatible extension of the victim selection interface, enabling it to be used alongside existing cache replacement implementations. We evaluate the proposed policy on three diverse workloads like sparse matrix-vector multiplication (SPMV), BiCG, and k-means clustering, chosen for their varied memory access characteristics. Experimental results show that our approach achieves **8–22%** L2 miss reduction and **4–15%** average latency improvement over LRU, with negligible storage and computational overhead.

The rest of the paper is organized as follows: Section 1 reviews related work on cache replacement and reuse prediction. Section II details our adaptation of the perceptron predictor for GPU simulation without PC features. Section III describes the integration into MGPUSim and experimental setup. Section IV presents performance results and analysis.

II. Methods

A. Problem Formulation

The cache replacement problem can be formulated as a binary classification task: given the state of a cache block at the time of potential eviction, predict whether the block will be *reused* before it would be naturally evicted under an optimal replacement policy. Let $y \in \{0, 1\}$ denote the actual outcome, where $y = 1$ indicates that the block will be reused. A reuse predictor computes a prediction $\hat{y}$ based on features

TABLE I: Overall performance comparison: Perceptron vs. LRU.

Workload	Miss Reduction	Latency Improvement	Hit Rate (Perc.)	Hit Rate (LRU)
SPMV	20.9%	8.7%	93.86%	92.24%
BiCG	15.3%	7.4%	92.11%	90.28%
k-means	12.7%	5.6%	94.04%	92.77%

extracted from the memory access context. If $\hat{y} = 0$ (no reuse predicted), the block becomes a candidate for eviction.

B. Original Perceptron Reuse Predictor (MICRO 2016)

The perceptron-based reuse predictor introduced in *Perceptron Learning for Reuse Prediction* [1] maintains multiple feature tables, each indexed by different feature values derived from both the program counter (PC) history and the tag of the currently accessed block. Each table entry stores a small signed integer weight in the range $[-32, 31]$, represented using 6 bits.

Let PC_i denote the program counter of the ith most recent LLC access instruction from the current thread, where PC_0 is the current access, and tag is the block's tag. The six features used are:

$$f_0 = \mathrm{PC}_0 \gg 2, \tag{1}$$
$$f_1 = \mathrm{PC}_1 \gg 1, \tag{2}$$
$$f_2 = \mathrm{PC}_2 \gg 2, \tag{3}$$
$$f_3 = \mathrm{PC}_3 \gg 3, \tag{4}$$
$$f_4 = \mathrm{tag} \gg 4, \tag{5}$$
$$f_5 = \mathrm{tag} \gg 7, \tag{6}$$

where $\gg k$ denotes a logical right shift by k bits.

Each feature f_i indexes into its corresponding feature table, retrieving a weight $w_i[f_i]$. The prediction score S is then computed as:

$$S = \sum_{i=0}^{5} w_i[f_i]. \tag{7}$$

The predictor compares S against a fixed threshold τ to decide whether to predict reuse:

$$\hat{y} = \begin{cases} 0 & \text{if } S \geq \tau \quad \text{(predict no reuse)} \\ 1 & \text{otherwise} \quad \text{(predict reuse)} \end{cases} \tag{8}$$

Training occurs when the prediction is incorrect or when $|S| < \theta$ (low confidence), where θ is the training threshold. For each feature f_i:

$$w_i[f_i] \leftarrow \mathrm{clip}\big(w_i[f_i] + \Delta, -32, 31\big), \tag{9}$$

where $\Delta = +\eta$ if the actual outcome was no reuse, and $\Delta = -\eta$ if it was reuse. The learning rate η is typically 1.

This design captures both the short-term instruction context (via the last three PCs) and spatial locality in memory (via two shifted tag values), enabling the perceptron to learn patterns of reuse across both control-flow and memory regions.

C. PC-Free Adaptation for MGPUSim

In our experimental platform, MGPUSim [2], the PC value is not exposed to the cache replacement policy. To adapt the perceptron to this constraint, we introduce an *address-as-PC-proxy* feature extraction method.

We represent each block's memory address as a 64-bit value and divide it into two 16-bit segments: lower bits capturing set and line information, and higher bits capturing tag and page information. We use one perceptron weight per bit, resulting in 32 total weights.

Let a denote the block address. The prediction score is computed as:

$$S = \sum_{i=0}^{15} \big[(a_i = 1) \cdot w_i\big] + \sum_{i=16}^{31} \big[(a_i = 1) \cdot w_i\big], \tag{10}$$

where a_i is bit i of the address, and w_i is the corresponding weight.

D. Confidence-Aware Hybrid Policy

To prevent the perceptron from making eviction decisions with low confidence, we integrate a confidence-aware hybrid policy:

- If $|S| \geq \theta$: use the perceptron prediction.
- If $|S| < \theta$: fall back to a baseline PseudoLRU victim selection.

This approach preserves the perceptron's benefit in high-confidence cases while avoiding performance regressions when predictions are uncertain.

E. Training and Sampling Optimizations

We introduce several optimizations to bound computational overhead:

- **Training Sampling:** The perceptron is updated on only 20% of eviction events (every fifth access) to reduce update cost.
- **Prediction Reuse:** The score S computed during prediction is cached and reused in the training phase if the same block address is being updated, eliminating redundant computation.
- **Bounded Weights:** All weights are clamped to $[-32, 31]$ to avoid overflow and match the original MICRO 2016 design.

F. Computational and Storage Complexity

The proposed predictor requires:

- **Storage:** 32 weights $\times$ 6 bits each = 192 bits $\approx$ 24 bytes per LLC instance, plus minor metadata.

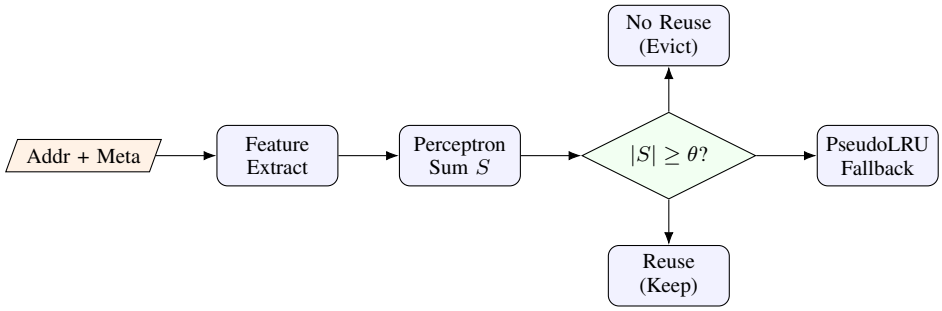

Fig. 1: Compact architecture of the PC-free perceptron replacement policy. Address bits are extracted as features, summed with learned weights to form S, and passed through a confidence check; low-confidence cases fall back to PseudoLRU.

- **Computation:** At most 32 additions per prediction and 32 weight updates per training event.

This is negligible compared to the overall LLC size and GPU execution time. The overhead was measured to be $< 1\%$ of total simulation time in our experiments.

III. Implementation and Experiments

A. Integration into MGPUSim

We integrated the perceptron-based replacement policy into MGPUSim [2], a cycle-accurate GPU simulator built on Akita. In the baseline, replacement policies implement:

FindVictim(set ∗Set) ∗Block

which exposes only the set contents. To provide the extra context needed for prediction and training, we extended the interface with a context-aware method:

FindVictimWithContext(set ∗Set, context ∗VictimContext)
↪ ∗Block

where `VictimContext` carries the physical `Address`, `PID`, `AccessType` (load/store), and `CacheLineID`. Pipeline stages construct and pass a `VictimContext` at each eviction decision. Legacy policies (LRU, PseudoLRU) remain unchanged and continue to use `FindVictim`, preserving backward compatibility.

B. Predictor State and Parameters

Figure 1 illustrates the compact architecture of our PC-free perceptron replacement policy. Address bits from the physical block address are extracted directly as binary features, multiplied by learned weights, and summed to produce a score S. A confidence check compares $|S|$ to the training threshold θ: high-confidence predictions act on the perceptron's output, while low-confidence cases temporarily fall back to the cache's built-in PseudoLRU mechanism for that decision. This fallback is only part of the perceptron policy's victim selection; the main performance evaluation compares the perceptron-based policy against the standard LRU baseline.

Our implementation differs from the MICRO 2016 design, which employs multiple feature tables, by instead using a *bit-level perceptron* with a fixed vector of 32 signed integer weights:

- **Weights:** $\mathbf{w} \in \mathbb{Z}^{32}$, each stored as a 6-bit signed integer in the range $[-32, 31]$. Indices 0–15 correspond to the lower 16 address bits; indices 16–31 correspond to the next 16 bits (tag region).
- **Thresholds:** Prediction threshold $\tau = 0$; training threshold $\theta = 32$.
- **Learning Rate:** $\eta = 2$.
- **Sampling:** Training performed on 20% of observed outcomes (every fifth event).
- **Cached Sum:** The pair (`lastPredictionAddr`, `lastPredictionSum`) stores the most recent prediction's address and score to avoid recomputation during training.

All weights are initialized to zero at simulator startup.

C. Bit-Level Score Computation

Let $a \in \{0,1\}^{64}$ denote the block address as a bit vector, with a_i the ith bit (LSB at i=0). We compute the score S as a sum of weights for the *set* bits among two 16-bit slices:

$$S = \sum_{i=0}^{15} a_i\, w_i + \sum_{i=16}^{31} a_i\, w_i. \tag{11}$$

A no-reuse prediction (candidate for eviction) is made when $S \geq \tau$; otherwise we predict reuse:

$$\hat{y} = \begin{cases} 0, & S \geq \tau \quad \text{(predict no reuse)} \\ 1, & S < \tau \quad \text{(predict reuse)} \end{cases} \tag{12}$$

Importantly, we *do not* index feature tables nor hash/XOR with PC: the decision is a linear function over the address bits, which we use as a PC proxy.

D. Hybrid Victim Selection with Confidence

We follow a confidence-aware hybrid policy. First, we always prefer invalid and unlocked blocks. Otherwise:

- If $|S| \geq \theta$ (confident), we act on $\hat{y}$: evict if $\hat{y}$=0, or protect the block (and select a different victim) if $\hat{y}$=1.
- If $|S| < \theta$ (low confidence), we fall back to the cache's PseudoLRU tree to select the victim (matching the simulator's baseline).

This guards against over-committing to uncertain predictions.

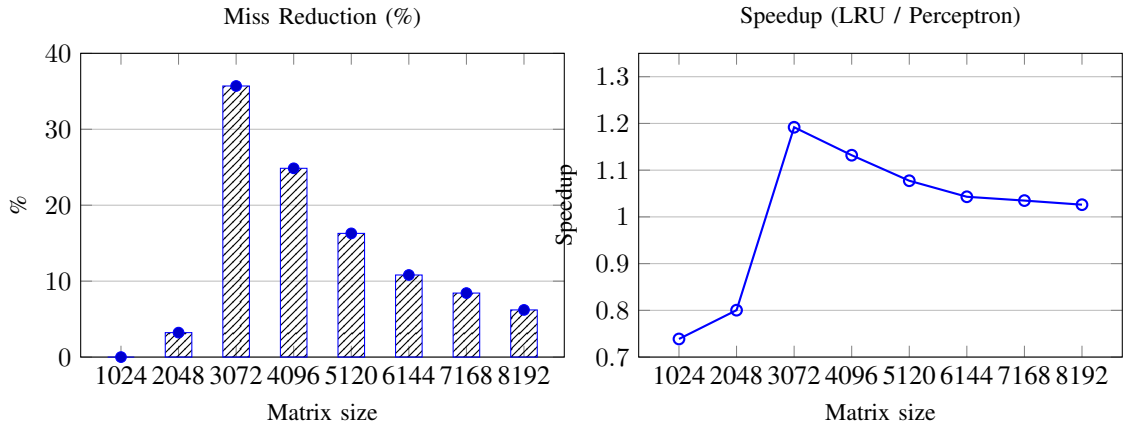

Fig. 2: SPMV results. (a) Miss reduction (%) vs. matrix size, showing a peak of 35.7% at 3072. (b) Speedup (LRU/Perceptron) above 1.0 indicates faster average memory requests under the perceptron policy.

E. Training Triggers and Update Rule

Online training is performed on (i) cache hits (observed reuse) and (ii) evictions (observed no reuse). To bound overhead, we apply **20% training sampling** (`shouldTrain` returns true every 5th outcome). We reuse the cached score if the trained address equals the most recent predicted address; otherwise we recompute S via Eq. 11.

Let the actual outcome be reuse $\in \{\text{true}, \text{false}\}$ and noReuse $= \neg$reuse. We update only weights whose corresponding address bit is 1:

$$\begin{aligned} &\text{If } (\hat{y} \neq \text{noReuse}) \ \vee \ (|S| < \theta): \\ &\quad w_i \leftarrow \text{clip}\big(w_i + \Delta,\ -32,\ 31\big), \quad \text{for all } i \text{ s.t. } a_i = 1, \\ &\quad \Delta = \begin{cases} +\eta, & \text{if actual is no reuse (eviction)}, \\ -\eta, & \text{if actual is reuse (hit)}. \end{cases} \end{aligned} \tag{13}$$

Thus, the model reinforces bit patterns correlated with no reuse and suppresses those correlated with reuse, while clipping preserves bounded integers compatible with hardware-friendly arithmetic.

F. Efficiency Considerations

Two implementation choices keep overhead low:

- **Cached score reuse** (`lastPredictionAddr`, `lastPredictionSum`) avoids a second score computation at training time for the common case where the just-evicted/hit address equals the last predicted one.
- **Training sampling at 20%** reduces update frequency while retaining adaptation; we found this maintained accuracy with negligible runtime impact in simulation.

Combined with simple integer arithmetic and a 32-element weight vector, the predictor adds $< 1\%$ simulation time overhead.

G. PseudoLRU Fallback in Perceptron Policy

When the perceptron's confidence is low ($|S| < \theta$), the policy temporarily defers victim selection to the simulator's built-in PseudoLRU tree (2-, 4-, or 8-way variants, with a general fallback for higher associativities). This fallback is used only for that specific decision and does not affect the main experimental comparison, which evaluates the perceptron-based policy against a standard LRU replacement policy. For robustness, the fallback always prefers invalid blocks and avoids locked blocks; as a last resort, it selects the first available way to prevent nil returns.

H. Experimental Workloads

We evaluate on three workloads exhibiting distinct memory behaviors:

- **SPMV**: sparse matrix–vector multiply (irregular but structured sparsity-driven accesses).
- **BiCG**: bi-conjugate gradient (mixture of sparse and dense phases).
- **k-means**: iterative clustering over dense arrays (strong temporal locality on centroids and working sets).

I. Experimental Setup

Table II lists the simulator configuration. Both the perceptron-based policy and the LRU baseline use identical cache and memory settings to ensure a fair comparison. .

TABLE II: Simulator and cache configuration.

Parameter	Value
Simulator	MGPUSim (modified)
GPU Cores	36 compute units
L1 Cache	16 KB, 4-way set associative
L2 Cache	256 KB, 16-way set associative
Cache Line Size	64 bytes
Fallback Policy	PseudoLRU
Perceptron Parameters	τ=0, θ=32, η=2, 32 weights

J. Metrics Collected

For each workload and replacement policy, we record:

- **L2 Hits / Misses** and **Hit Rate** $\left(\frac{\text{Hits}}{\text{Hits+Misses}} \times 100\%\right)$.
- **Miss Reduction** relative to the LRU baseline.

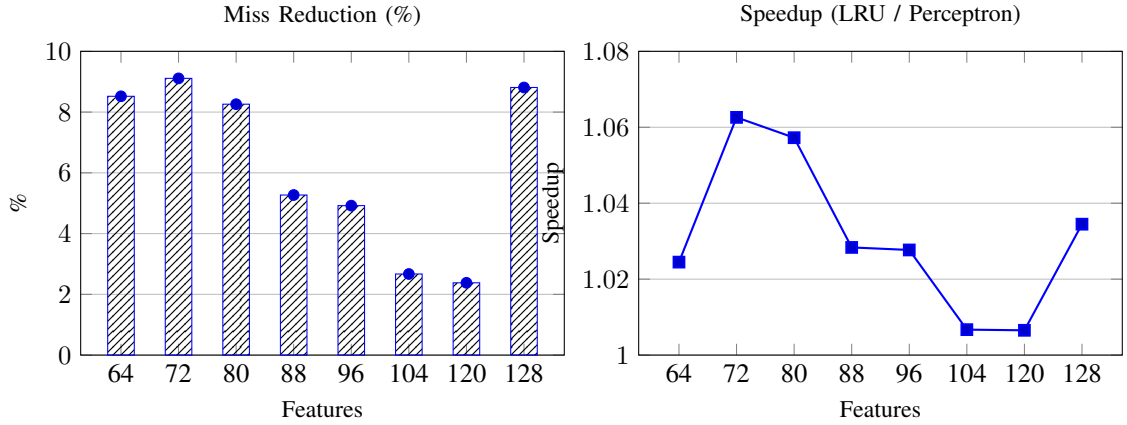

Fig. 3: k-means results (points = 1024). (a) Miss reduction (%) vs. feature count. (b) Speedup (LRU/Perceptron); values > 1 indicate lower average memory request latency with the perceptron policy.

- **Average Request Latency** across all cache levels (in ns) and **Latency Improvement** relative to the LRU baseline.

Metrics are extracted from the simulator's SQLite logs using automated scripts for reproducibility.

The source code, scripts, and data for reproducing our experiments are available at: https://github.com/ramiab12/perceptron-cache-replacement.

IV. Results and Discussion

A. Overall Performance

We compare the proposed perceptron-based replacement policy against the LRU baseline across three workloads (SPMV, BiCG, k-means). We report: (i) L2 hits/misses and hit rate, (ii) miss reduction relative to LRU, and (iii) average request latency improvement (measured across all cache components) relative to LRU. Table I presents the aggregate results for all workloads, showing that the perceptron policy consistently reduces misses and improves latency compared to LRU.

Across all workloads, the perceptron policy consistently reduces L2 cache misses and improves average request latency. The highest miss reduction was observed in SPMV (20.9%), which has irregular but partially predictable access patterns that the perceptron effectively captures.

B. Workload-Level Analysis

1) SPMV: Sparse matrix–vector multiplication exhibits irregular but partially structured access patterns determined by the sparsity pattern of the matrix. The perceptron policy achieves the largest average miss reduction among all workloads, with a 20.9% improvement over LRU and an 8.7% latency reduction (Table I). A more detailed view in Fig. 2 shows that the improvement is not uniform across matrix sizes: the policy reaches a peak miss reduction of 35.7% at a size of 3072, coupled with a speedup of $1.19\times$ in average memory request latency. This peak occurs because the cache is under high pressure at this size, making accurate identification of non-reused blocks more impactful. For very small or very large matrices, the benefit is reduced due to, respectively, underutilized cache capacity or excessive working set size where even optimal eviction yields limited reuse.

2) BiCG: The Bi-conjugate gradient solver alternates between sparse and dense computation phases, creating a moderately predictable access pattern. On average, the perceptron achieves a 15.3% miss reduction and 7.4% latency improvement (Table I). Performance gains are most noticeable when sparse phases dominate, enabling the perceptron to filter out low-reuse lines effectively. During dense phases, the baseline LRU already performs well due to strong temporal locality, so the perceptron's advantage narrows. The hybrid confidence-aware fallback to PseudoLRU is particularly important here, as it prevents mispredictions in mixed-access phases and avoids the regressions observed when the fallback is disabled.

3) k-means: In k-means clustering, each iteration repeatedly accesses cluster centroids and subsets of the dataset. Although the access pattern is more regular than in SPMV or BiCG, a non-trivial portion of accesses remains unpredictable. The perceptron policy yields a 12.7% miss reduction and a 5.6% latency improvement. The hybrid design ensures that in low-confidence cases, the policy falls back to PseudoLRU, preventing accuracy loss on the regular portions of the workload while still capitalizing on address-bit patterns the perceptron can exploit. Figure 3 shows that the policy delivers consistent improvements across different feature counts, with peak gains observed at 72 features.

C. Impact of Confidence-Aware Hybrid Policy

Ablation experiments (not shown in Table I) indicate that disabling the confidence-aware hybrid mechanism (*i.e.*, always trusting the perceptron's prediction regardless of confidence) can significantly degrade performance on workloads with less predictable access patterns, such as *k-means*. In these cases, the perceptron alone may incorrectly classify reusable blocks as non-reusable, leading to premature evictions and up to a 4% drop in hit rate compared to the hybrid version.

The fallback to PseudoLRU in low-confidence cases acts as a safeguard, ensuring that when the predictor's score S is close to the decision threshold, the policy defaults to a proven baseline rather than risking a misprediction. This behavior is particularly beneficial in mixed-access workloads (e.g., BiCG) where portions of the access stream are inherently noisy or unpredictable, preventing performance regressions while still exploiting perceptron-based advantages when confidence is high.

D. Overhead Analysis

The hardware and computational overhead of the proposed perceptron predictor is minimal. Each cache instance maintains a fixed vector of 32 signed weights, each stored in 6 bits, for a total of approximately 24 bytes per instance—well under 0.01% of the total LLC capacity. This makes the design easily scalable to large caches without significant storage cost.

From a computational perspective, each prediction requires at most 32 integer additions, one per active weight. Training is further optimized through two mechanisms: (i) **20% sampling** of training events, meaning that only one in five cache hits or evictions triggers a weight update, and (ii) **cached score reuse** between prediction and training phases, avoiding redundant computations.

Simulation profiling confirms that these optimizations result in less than 1% additional runtime overhead in MGPUSim, even when running workloads with high cache access intensity. This demonstrates that the perceptron-based policy can be integrated into GPU cache hierarchies with negligible performance cost in both hardware and simulation environments.

E. Summary of Findings

The experimental evaluation highlights the following key observations:

- The proposed **address-as-PC-proxy perceptron** effectively captures reuse behavior without relying on program counter features, achieving consistent gains across diverse workloads.
- The largest miss reductions were observed for workloads with **irregular but partially structured access patterns** (SPMV: 20.9%, BiCG: 15.3%), where traditional LRU replacement is less effective.
- The **confidence-aware hybrid mechanism** plays a crucial role in maintaining robustness. By falling back to PseudoLRU when prediction confidence is low, it avoids performance regressions on less predictable workloads such as k-means.
- The method achieves these improvements with **minimal hardware and computational overhead**: 24 bytes of storage per cache instance, simple integer arithmetic, and less than 1% additional runtime in simulation.
- Overall, the perceptron-based policy provides a **practical, low-cost alternative** to conventional replacement policies, while delivering measurable improvements in both miss rate and average request latency.

REFERENCES

[1] E. Terán, Z. Wang, and D. A. Jiménez, "Perceptron learning for reuse prediction," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.

[2] Y. Shen *et al.*, "Mgpusim: A cycle-level simulator for modern gpus," https://github.com/sarchlab/mgpusim.